Held fee classification, as a round trip: download, classify, re-upload

Closing the loop on fee_gl_unassigned without a second, parallel way of tying a fee to a loan

16 October 2026

Dupleix Institute, for the build team, MAIIC Finance, and external reviewers (Codex, Copilot)

Scope note, revision 3. Next build item after the GL fee route (spec v4 sections 7.8.3 to 7.8.5).

What this spec actually buys MAIIC

124 fee-GL postings — arrangement and legal fees E-Banker could not automatically tie to a loan — sit unresolved today, growing every month, and each one is a loan whose EIR cannot safely be locked until it's settled (section 1.3). Right now the only way to resolve one is a single click in a screen, one row at a time. This item lets Finance clear that backlog at volume, in a spreadsheet, **without opening a second way to write a fee into the system**: every real decision in the file still calls the exact same governed operation the screen already calls (`assignHeld()` or `HeldFeeEvidence::dismiss()`), so the control that stops a fee being counted twice (section 1.3) is never bypassed for the sake of throughput. What it is not: a faster way to guess — a row the automated matching rules (section 1.2) couldn't settle is still settled by a person looking at the evidence, just at the pace a spreadsheet allows rather than one row per page load.

0. Note to the reviewer

This note is written for someone reviewing the design with no prior exposure to this codebase or to the conversation that produced it. Section 1 is a primer on the business and the data it works with; section 2 explains the specific mechanism this build item extends; sections 3 onward are the actual scope. If you already know MAIIC's E-Banker data model, skip to section 3.

Revision history.

- **Revision 1 (16 October 2026):** Copilot's review — `docs/reviews/Copilot_Review_Held_Fee_Classification_Export_Reimport_2026-10-16.md` — found that an unchanged, re-uploaded export had no defined no-op outcome, plus gaps in the file-format contract, deferral-note provenance, and acceptance coverage. Folded into the draft that followed.
- **Revision 2 (16 October 2026):** Codex's review — `docs/reviews/Codex_Review_Held_Fee_Classification_Export_Reimport_2026-10-16.md` — read the actual services (`GlFeeBuildService`, `HeldFeeEvidence`, the controller) and found that revision 1 made claims the code does not support: `held_id` **is wiped and reissued on every rebuild**, `dismissCandidate` **does not mean "not a loan" or "Mega Farm,"** `tieHeld`**'s only check is a 10-character reason and account existence — there is no batch/amount re-check to reuse,** and **a TIE reason cannot be optional, because the shared service itself requires one.** This revision corrects all four and adds the identity, concurrency, and durability handling Codex specified. Where revision 1 and Codex disagreed, Codex wins — it is reading the code, not reasoning about it secondhand.
- **Revision 3 (16 October 2026, this one):** a direct audit-completeness review of revision 2 found that successful decisions are logged (they already reuse `assignHeld()`'s and `dismiss()`'s existing `AuditLoggerService` calls), but **failed, stale, conflicting, and rejected attempts were not logged at all**, and **nothing recorded that a tie came from a file upload rather than the screen** — a direct gap against

Codex's own finding 5 ("keep file provenance in separate audit metadata"), never actually closed in revision 2. Section 6 now adds an upload-level audit entry and an `$origin` parameter threaded through the two reused operations.

1. Background: what this system is, and the data it reads

MAIIC is a microfinance/cooperative lender. **E-Banker** is its core banking system (an Oracle database, supplied by a vendor called Virtual Galaxy), where every loan account, every posting, and the general ledger live. The system this spec belongs to is a separate IFRS 9 engine that Dupleix Institute is building for MAIIC: it reads data **out of** E-Banker (never writes to it) and calculates expected credit losses, the effective interest rate (EIR) on each loan, and the interest revenue MAIIC should recognise under IFRS 9. None of what follows changes E-Banker; it is all about getting E-Banker's own data correctly attributed inside this separate system.

1.1 How a loan fee actually gets posted in E-Banker

When MAIIC disburses a loan, it deducts certain fees from the amount paid out — an arrangement fee and a legal fee are the two this spec is about (others exist: appraisal, insurance, a loan guarantee fee). E-Banker records this as **one balanced double-entry batch**. A worked example, Malawi Police SACCO, 7 May 2025:

Leg	Account	Dr/Cr	Amount
The loan account, 000104430000072	the borrower's own loan GL	Debit (the amount the loan is carried at)	1,046,210,000
Bank	a cash GL	Credit (cash actually paid out)	1,000,000,000
4873 Arrangement fees	the fee income GL	Credit	30,000,000
4871 Legal fees	the fee income GL	Credit	16,210,000

The fee appears on **two legs of the same batch** — a credit to the income GL, and (either folded into the loan's gross debit, or as its own separate debit line, narrated e.g. "To Trf Arrangement fees") on the loan account. Roughly 56 percent of fees get their own separate debit line on the loan account; the rest are folded into one gross drawdown line. Either way, the loan is carried gross of fees and the fee income is credited once, in the same batch.

The fee income GLs this system cares about:

GL	What it is
4871	Legal fees
4873	Arrangement fees
4874	Insurance fees
4875	Guarantee fees

1.2 The trap: a fee-GL posting does not carry the borrower's account number

E-Banker models every GL, including an income GL like 4873, as if it were itself an "account" with its own account number — a fixed control-account number per GL (e.g. every 4873 credit in a given extract carries the same pseudo-account, not the borrower's). So an extract of "every posting to 4871 and 4873" tells you the date, the amount, and the narration of each fee — but **not** which borrower it belongs to, because the account number on that row is the GL's own control account, not the loan.

The borrower's account is on the **other leg of the same batch** — the loan-side debit described in 1.1. The only way to recover it is to join the fee-GL leg to the loan-side leg of the same batch, matching on the transaction date, the batch type, and the batch number (E-Banker's own posting reference), not on anything read off the fee leg itself.

This is why the system reads **two** extracts, not one:

- **GL_04a**: every posting to the fee income GLs (4871/4873/4874/4875) — the fee leg. Each row's own key is E-Banker's `CUMVOUCH_DET_ID` (its "voucher" number).
- **GL_04b**: every other leg of the same batches — potentially the loan-side leg, among others.

`GlFeeBuildService` (`app/Services/Ebanker/GlFeeBuildService.php`) joins them on (transaction date, batch type, batch number) and decides, for each fee posting, which loan it belongs to:

1. **One loan account in the batch** → the fee is that loan's. The simple case.
2. **Several loan accounts in the batch** → the loan whose own debit in the batch equals the fee's amount; failing that, the loan whose account name the fee's narration mentions. Anything else is held — never guessed from a partial match.
3. **No loan account in the batch, but the batch clears money out of GL 2010 ("Working Capital Loan")**, and a separate extract (GL_07) shows exactly one loan drawn into 2010 by that same amount on or before the fee's date → the fee is that loan's. Example: in October 2024 a drawdown was posted as loan debit / 2010 credit, then a *second*, later batch paid 2010 out to the bank net of the fee and credited the fee GL. Two real cases this rule resolved: Mafeni (4,250,000) and Agwenda (1,205,000 and 1,500,000).
4. **None of the above, or the batch belongs to the Mega Farm programme** (a separate book MAIIC administers, out of scope for the EIR engine under decision D30) → **held**: written into `fee_gl_unassigned`, with the reason, never dropped and never guessed.

A fee is material if missed: before this route was built, 180.8 million of 2025 fee income on eight loans was simply missing from the engine, and six of those loans had already had their EIR *locked* with no fee in it at all.

The one fact that drives most of this spec's identity design: `build()` rebuilds the whole held queue from scratch on every run — `DB::table('fee_gl_unassigned')->delete()`, then reinsert (`app/Services/Ebanker/GlFeeBuildService.php:126-139`). The table's auto-increment `id` is a *display* convenience, regenerated every rebuild. The only thing that identifies the *same real posting* across rebuilds is `external_transaction_id` — literally `'GL_04a:' . CUMVOUCH_DET_ID`, E-Banker's own voucher number. Anything this spec keys on `id` alone is unsafe the moment a rebuild runs between export and re-import, which happens routinely (the GL fee build reruns on a schedule and whenever new E-Banker packs land).

1.3 Where a resolved fee ends up, and why "the same fee twice" is a real risk

Every fee the system knows about — however it arrived — lands in one table, `contract_fees`, one row per fee per loan (`contract_id` is the E-Banker loan account number). But a fee can reach this table from **more than one source**: Finance's own workbook, the loan-origination system's capture at application, and now this GL-posting route. If the same fee is loaded twice from two sources, the engine's net investment calculation (drawn amount minus integral fees received) double-counts the deduction and the EIR comes out wrong — overstated, in the direction that matters. A governed setting (`fee_source_precedence`) decides which source is the one of record for a given loan and fee type; the losing source's row is marked `superseded_by_source`, kept for visibility, but never used in a calculation. **This is why this spec's re-import must call the system's existing, governed operations — `GlFeeBuildService::assignHeld()` and `HeldFeeEvidence::dismiss()` — rather than writing to `contract_fees` directly: a second write path is exactly the kind of duplicate source that causes this problem.**

1.4 What a held row actually is, and what the existing actions actually do

As at 16 October 2026, 124 fee-GL lines across the whole window these extracts cover (1 November 2024 to 23 September 2026) sit in `fee_gl_unassigned`. Each row keeps the posting itself (date, batch, GL, amount, narration), the reason it's held, every loan account that *was* in the batch (`loans_in_batch`), and — if the build has a best guess — a single `candidate_contract_id` with `candidate_evidence` explaining why.

Two actions exist today, in `EirFeeClassificationController`, and **this is what each one actually does — verified against the code, not assumed:**

- `tieHeld($held, contract_id, reason)` → calls `GlFeeBuildService::assignHeld()` (`app/Services/Ebanker/GlFeeBuildService.php:209-238`). This method: requires `reason` to be at least 10 trimmed characters; looks up the held row by its (ephemeral) numeric id; requires the given `contract_id` to exist in either `contract_eir` or `loan_books` (plain existence, nothing about whether it was in the batch, nothing about amount); then, in one transaction, imports the fee into `contract_fees` via `FeeImportService`, deletes the held row, writes an audit entry, and re-applies fee source precedence. **There is no re-run of the automated batch/amount matching rules from section 1.2.** This is a governed human override on recorded evidence — appropriate, since these rows are held *because* automation could not settle them — but it is not the stronger "consistency check" an earlier draft of this spec claimed.
- `dismissCandidate($held, contract_id, reason)` → calls `HeldFeeEvidence::dismiss()` (`app/Services/Eir/HeldFeeEvidence.php:57-80`). This requires the given `contract_id` to equal the held row's *current* `candidate_contract_id` exactly, then writes a row to `fee_gl_candidate_dismissals` keyed by `(external_transaction_id, contract_id)`. **The held row is not deleted and the posting is not excluded from anything.** Its only effect is: that specific candidate no longer blocks *that specific loan's* EIR from being solved (readiness checks consult the dismissal to stop treating the candidate as an open question for that loan). The posting itself stays in `fee_gl_unassigned`, unresolved, exactly as before. **There is no concept in this codebase, anywhere, of a held posting being terminally excluded as "not a loan's fee" or "Mega Farm."** A posting reaches a Mega Farm classification only through rule 4 in section 1.2, decided by the automated build itself from the batch evidence — never by a person overriding it.

Both actions require the `eir.govern` permission and are written through `AuditLoggerService`.

2. What this build item is, in one paragraph

124 held rows, growing every month, can currently only be resolved one click at a time in a screen, and only via `tieHeld` or `dismissCandidate` as they actually behave (section 1.4) — not as an earlier draft of this spec imagined them to behave. This item adds a **download** of the held queue, identified by the posting's durable `external_transaction_id` rather than its rebuild-prone numeric id, and a **validated re-upload** of Finance's decisions, where every accepted decision calls one of those same two operations, plus one small new durable note for deferrals (section 6). Nothing about how a fee gets tied to a loan changes; only how many rows a person can resolve per sitting, and how safely a round trip through a spreadsheet can be done at all given that the held queue itself is rebuilt from scratch on a schedule.

3. What this is not

- **Not a new E-Banker extract.** `customer_name` is already landed on `loan_books` per `contract_id`, from the account extract the system already pulls every month. Adding it to the export is a join against data already held, not new scope, not a new dependency on MAIIC ICT.
- **Not a new matching rule.** `GlFeeBuildService`'s three automated rules (section 1.2) are untouched. This only gives a person a faster way to work through the cases those rules leave held.
- **Not a second way to tie a fee, or to dismiss a candidate.** Every accepted row in the re-import calls `assignHeld()` or `HeldFeeEvidence::dismiss()` exactly as the screen does (section 1.4) — which matters because of the double-counting risk in section 1.3.
- **Not a terminal "this posting is not a loan's fee" or "this is Mega Farm" disposition.** No such governed mechanism exists today (section 1.4). Inventing one — durable storage, audit, interaction with rebuilds and readiness checks — is a real feature in its own right and is explicitly **out of scope** for this item (section 9), not something this spec can safely bolt onto `dismissCandidate`.

4. customer_id and customer_name: what was decided, and why

Dropped `customer_id`. The account number (`contract_id`) is already the key, per the 7 October 2026 decision that the E-Banker system record — not a signed letter or any other document naming the borrower — governs the EIR calculation (recorded as decision O19 in the EIR engine specification). A second identifier alongside it adds nothing a reviewer or Finance user needs.

Kept `customer_name`, but strictly as a label, never a match key. Shown next to a candidate account so a person can recognise it at a glance — some narrations already name the borrower ("By Trf Insurance fees for Milele Agroprocessing Limited Loan"), but not all do, and for the ones that don't, a bare account number tells a reviewer nothing. This mirrors how the *existing* fee workbook import already treats a name: `ImportSpecs` (`app/Support/Imports/ImportSpecs.php`, the `fees` entry) already allows "its customer's name" as one of three ways a fee workbook line is accounted for against a GL posting, alongside the amount and the date — so showing a name here is consistent with an idea the system already uses, not a new one.

The risk, and why "label only" is the mitigation. A confident-looking name next to an *unconfirmed* candidate account invites a person to agree with the guess because the name looks right, rather than checking the evidence. The same account-number-over-name reasoning from O19 applies directly: keeping the name a label — never read by the import, never part of what is validated, proven by acceptance 8.6 — keeps the account number as the only thing actually decided.

5. Export

Route: `GET eir-fee-classification/held/export`, permission `settings|eir.fees.import` — the same permission the index already requires to read the queue. Must be registered as its own route with its own middleware group (the controller's constructor applies `settings|eir.fees.import` only to `index` today — the new export route needs the same grant explicitly, it does not inherit it).

Source: `fee_gl_unassigned` (current rebuild), left joined to `loan_books` on `candidate_contract_id` (latest `reporting_period`) for `customer_name`, and left joined to `fee_gl_candidate_dismissals` on `(external_transaction_id, candidate_contract_id)` to show whether today's candidate was already rejected on a previous rebuild (section 1.4 — a rebuild can re-propose a candidate that was already dismissed, since the dismissal table and the automated `candidate()` logic are independent of each other).

One row per currently-held posting, current state, not a history.

Column	From	Notes
`external_transaction_id`	`fee_gl_unassigned.external_transaction_id`	**the identity the re-import matches on** — stable across rebuilds (section 1.2)
`held_id`	`fee_gl_unassigned.id`	shown for a person's convenience only; **never used by the import** — it is reissued on every rebuild
`transaction_date`, `batch`, `gl_code`, `amount`, `narration`	same	the posting itself (section 1.1); re-read at import time as the evidence fingerprint (section 6)
`reason`	same	the reason the automated build (section 1.2) is currently holding it
`loans_in_batch`	same, flattened	every account that batch did carry, for reference
`candidate_contract_id`	same	the build's own best guess, if it has one, as of *this* rebuild
`candidate_customer_name`	`loan_books.customer_name` for that candidate	**label only** (section 4)

Column	From	Notes
`candidate_as_of_period`	the `loan_books.reporting_period` the name lookup used	so Finance can tell how current the label is
`candidate_evidence`	same	why the build thinks so
`candidate_previously_dismissed`	`fee_gl_candidate_dismissals` match on `(external_transaction_id, candidate_contract_id)`	true means a person already said this exact candidate is wrong, on an earlier rebuild — stops the same candidate being rejected again and again without Finance realising it already was
`deferral_note`, `deferral_note_by`, `deferral_note_at`	the new durable notes store (section 6)	the latest `DEFER` note recorded against this `external_transaction_id`, if any — survives rebuilds; blank if none
`contract_id`	*(blank)*	Finance fills this in for `TIE` or `CANDIDATE_REJECT`
`decision`	*(blank)*	blank is a valid, explicit **no-op** (section 6, rule 0); otherwise `TIE`, `CANDIDATE_REJECT`, or `DEFER`
`reason_note`	*(blank)*	free text, **required and at least 10 characters for every non-blank decision** — matching the 10-character minimum `assignHeld()` and `dismiss()` already enforce (section 1.4); there is no "optional" reason for any decision

File format: CSV, UTF-8, comma-delimited, RFC 4180 quoting. `external_transaction_id`, `held_id`, `contract_id`, `candidate_contract_id` and every entry in `loans_in_batch` are written and read as **text**, not numbers — leading zeroes in an E-Banker account number are significant, and spreadsheet software left to auto-detect the column type will strip them or render a long account number in scientific notation. `transaction_date` and `candidate_as_of_period` are ISO 8601 (`YYYY-MM-DD`). `amount` is decimal, two places, no thousands separator. `loans_in_batch` is a single cell, accounts joined by `;` (a character that cannot appear in an account number); an empty batch is an empty string, not omitted. `narration` may itself contain commas, quotes or newlines and must be read as a single quoted field, not split on them, and any value beginning with `=`, `+`, `-` or `@` is written with a leading apostrophe or otherwise escaped so a spreadsheet application cannot interpret it as a formula on open. The export and import both pin these rules under one version (`held_fee_classification_file_v1`) named in a header row/comment, so a future format change is visible rather than silently incompatible.

6. Re-import

Route: `POST eir-fee-classification/held/import`, permission `eir.govern` — the same permission `tieHeld()` and `dismissCandidate()` already require, since this is a bulk front end to those same governed operations and the new deferral-note write (which is also audit-logged, section 1.4's pattern applied to a new, smaller fact).

New service `HeldFeeClassificationImportService`, shaped like the existing `FeeImportService::import()` (`app/Services/Eir/FeeImportService.php`): not silent, every row accounted for in the result, with one of these outcomes: `unchanged`, `tied`, `candidate_rejected`, `deferred`, `rejected`, `already_resolved`, `stale_evidence`, `not_currently_held`.

Identity and staleness, resolved per row, before any decision logic runs:

1. Re-read the **current** held row from `fee_gl_unassigned` by `external_transaction_id` — never by `held_id`, which may belong to a different posting entirely after a rebuild (section 1.2).
2. If no current held row has that `external_transaction_id`: look it up in `contract_fees` by `source_system = 'E-Banker GL fee posting' AND external_transaction_id = ...` (`GlFeeBuildService::SOURCE`, the same constant `assignHeld()` writes). Found → `ALREADY_RESOLVED` (someone, by any route, already tied it; no write).

Not found anywhere → `NOT_CURRENTLY_HELD`, reported distinctly and not treated as a success — the posting may have been reversed upstream, or something else needs investigating; the import must never guess.

3. If a current held row **is** found: compare the file's exported `amount`, `gl_code`, `transaction_date`, `batch` and `candidate_contract_id` against the current row's own values. Any difference → `STALE_EVIDENCE`: the evidence Finance reviewed is not the evidence that exists now (a rebuild can change the candidate, or — rarely — the underlying extract can correct a figure), so the decision is refused rather than applied against evidence nobody actually looked at. A fresh export is required before that row can be decided.
4. Only once the row is confirmed current and unchanged does any decision get evaluated.

Field whitelist: once past identity resolution, the decision logic reads exactly three columns — `contract_id`, `decision`, `reason_note`. Every display column (`narration`, `candidate_customer_name`, `candidate_as_of_period`, `candidate_evidence`, `candidate_previously_dismissed`, `loans_in_batch`, `reason`, the deferral columns) is present for Finance's evidence while deciding and is read only for the staleness check in rule 3 above, never to choose or validate the decision itself. Acceptance 8.6 proves editing or removing `candidate_customer_name` specifically has no effect.

Rule 0 — all three editable cells blank is `UNCHANGED`, not a rejection. A row with `decision`, `contract_id` and `reason_note` all blank, exactly as the export leaves them, is an explicit no-op: nothing is read from `fee_gl_unassigned` beyond the staleness check, nothing is tied, rejected, or deferred, no audit entry is written, and the outcome is `UNCHANGED`. Any *other* blank/filled combination — a filled `contract_id` or `reason_note` with `decision` blank, for instance — is `REJECTED` as inconsistent, never silently reinterpreted as a different decision. This is what makes "export the whole queue, edit a handful of rows, upload the whole file back" a safe default workflow, and is what acceptance 8.1 (an untouched re-upload) tests.

The three decisions, each calling an existing governed operation exactly as the screen does:

- `TIE` — requires `contract_id` and `reason_note` (trimmed, ≥10 characters, ≤500, matching the screen's own limit). Normalises `contract_id` with `App\Support\ContractId::normalise`. Calls `GlFeeBuildService::assignHeld($heldId, $contractId, $userId, $reasonNote)` — the *current* row's numeric id (resolved in the identity step above, not the file's `held_id`). This is exactly `tieHeld()`'s own check (section 1.4): account existence in `contract_eir` or `loan_books`, nothing stronger is invented here.
- `CANDIDATE_REJECT` — requires `contract_id` and `reason_note` (same length rule). The given `contract_id` must equal the *current* row's `candidate_contract_id` — if it doesn't (including if the row now has no candidate at all), the row is `REJECTED` with that reason, because `HeldFeeEvidence::dismiss()` itself requires the exact match (section 1.4) and this import must not pre-empt or soften that check. On success, calls `HeldFeeEvidence::dismiss()`. The posting **stays held** — this decision narrows a candidate for one loan, it does not resolve the posting, and the result/screen must say so plainly so Finance does not mistake it for the row being done.
- `DEFER` — requires `reason_note` only (`contract_id` must be blank; filled is `REJECTED`). Writes (upserts) the note, author and timestamp to a new table, `fee_gl_held_notes` (`external_transaction_id` unique key, `note`, `author_id`, `created_at/updated_at`), replacing any earlier note for that posting. Audit-logged via `AuditLoggerService` the same as every other decision here. This table is new, small, and **not** part of `fee_gl_unassigned`, specifically so a rebuild of the held queue never erases it (section 1.2).

Duplicate `external_transaction_id` within one file: rejected, deterministically, for *every* row sharing that identity — never "first wins" or "last wins" decided by file order.

Concurrency and atomicity. Per row, from the identity re-read (step 1 above) through the decision write, the held row is locked (`lockForUpdate()`) inside one transaction, so a concurrent screen action, a concurrent second import, and a rebuild running mid-import cannot interleave with this row's decision. Within that lock:

- A `TIE` that finds `contract_fees` already holds this `external_transaction_id` (the unique source/external-id constraint — `database/migrations/2026_08_03_000000_strengthen_eir_fee_classification.php` — or

`FeeImportService`'s own skip-on-repeat) tied to the **same** account the file requests is reported `already_resolved`, idempotently, not as a fresh tie. Tied to a **different** account is a `CONFLICT`, reported as such, never silently accepted as a success.

- Fee-source precedence is re-applied, as `assignHeld()` already does, inside the same transaction boundary it uses today; if that step fails after the fee itself has been written, the row's result is reported as a distinct `partial` outcome (fee tied, precedence not yet reapplied) rather than a clean success, and recovery is the next scheduled GL fee build run, which re-applies precedence globally — no bespoke replay mechanism is being built for this item.
- An unrelated row's infrastructure failure (a timeout, a deadlock on a different row) is reported for that row only and does not abort rows already decided or still to be decided in the same file.

Execution model, stated because it determines who an audit entry is attributed to: the import runs synchronously inside the uploader's own authenticated request, never a queued job. `AuditLog` (`app/Services/AuditLoggerService.php`) attributes every entry to `auth()->id()` from the current request — correct only while the upload is processed in-request. If a later performance need pushes this onto a queue, the real uploader's id must be passed through explicitly and used in place of `auth()->id()`, or every entry from that path silently attributes to a worker identity instead of a person; that is a condition on building it that way, not an incidental detail.

6.1 Audit completeness

Revision 2 reused `assignHeld()`'s and `dismiss()`'s existing audit calls as-is for a successful `TIE` or `CANDIDATE_REJECT`, which is correct — those are already logged, with old/new values, the reason, and the user, exactly as a screen-driven decision is. Two things were still missing, found on direct review of what does and doesn't call `AuditLoggerService` in the design: nothing recorded a failed, stale, conflicting, or rejected attempt, and nothing distinguished a file-driven tie from a screen-driven one in the log itself. Both are closed here, additively, without changing how the screen's own calls behave:

1. **One upload-level audit entry per import, always, regardless of outcome.** Logged once, synchronously, before the per-row results are returned: `AuditLoggerService::log('Held Fee Import Processed', 'fee_gl_unassigned', null, ['new_values' => ['file_name' => ..., 'file_checksum' => sha256 of the uploaded content, 'row_count' => ..., 'outcomes' => ['unchanged' => n, 'tied' => n, 'candidate_rejected' => n, 'deferred' => n, 'rejected' => n, 'already_resolved' => n, 'stale_evidence' => n, 'not_currently_held' => n, 'conflict' => n], 'rows' => [per-row: external_transaction_id, decision requested, outcome, reason if rejected/stale/conflict]], 'meta' => ['user' => $userId]])`. This is the one place a `CONFLICT` or a `STALE_EVIDENCE` becomes a durable fact rather than a message the uploader's screen showed once and lost — both are control-relevant events (the first means two attempts disagreed about which loan owns a posting; the second means a rebuild changed the evidence under a pending decision), and an auditor should be able to find every one of them without relying on anyone having been watching the screen at the time.
2. **An `$origin` parameter on the two reused operations, additive and optional.** `GlFeeBuildService::assignHeld(int $heldId, string $contractId, int $userId, string $reason, ?array $origin = null)` and `HeldFeeEvidence::dismiss(int $heldId, string $contractId, string $reason, int $userId, ?array $origin = null)` each merge `$origin` into the `meta` of their existing `AuditLoggerService::log()` call when it is given, and are unchanged when it is not — so the screen's own calls (which never pass it) are byte-for-byte unaffected. The import passes `['channel' => 'bulk_import', 'import_batch_id' => <the id of the upload-level entry above>]`. After this, a `TIE` or `CANDIDATE_REJECT`'s own audit entry — the one that already existed — says whether it came from the screen or a specific upload, and which one.

Acceptance 8.17–8.19 (below) cover this directly.

7. Screen

No new page. The existing Fee Classification screen gets a download button for the export, and an upload control that shows the per-row result table (the outcomes from section 6) once a file is submitted — nothing silent, nothing that reports success for a row it did not actually resolve, and a visible distinction between "tied" (posting resolved) and "candidate rejected" (posting still held, one candidate narrowed) so a person does not mistake the second for the first.

8. Acceptance

1. Exporting the current held queue and re-uploading it **unchanged** reports every row `UNCHANGED` (rule 0) — no tie, no candidate rejection, no deferral note written, and no audit entry for any row.
2. Filling `contract_id`/`TIE` with a ≥10-character `reason_note` for a confident candidate, and re-uploading, moves that row out of `fee_gl_unassigned` into `contract_fees` (section 1.3), with an audit entry, exactly as `tieHeld()` would — and the row is gone from the *next* export.
3. `CANDIDATE_REJECT` with a `reason_note`, where `contract_id` matches the row's current candidate, records the dismissal and leaves the row **in** `fee_gl_unassigned`; the next export shows `candidate_previously_dismissed = true` for that posting if the rebuild re-proposes the same candidate.
4. `CANDIDATE_REJECT` where `contract_id` does **not** match the row's current candidate (including a row with no candidate) is `REJECTED`, naming the mismatch — never silently substituted or applied to a different candidate.
5. `DEFER` with a `reason_note` writes to `fee_gl_held_notes`; a rebuild of `fee_gl_unassigned` (simulated in the test) does not erase it, and the next export's `deferral_note` columns show it unchanged.
6. **Rebuild between export and re-import:** export a row, run the GL fee build again (changing nothing upstream) so the row gets a new `held_id`, then upload a `TIE` for it using the *original* file. The import resolves it correctly by `external_transaction_id` despite the `held_id` mismatch.
7. **Stale evidence:** export a row, then (in the test) change its `candidate_contract_id` via a rebuild before uploading a decision keyed to the original candidate. The row is `STALE_EVIDENCE`, not applied, not guessed.
8. A row already tied by hand in the screen (or by a previous import) after the file was downloaded reports `ALREADY_RESOLVED` on re-upload, not an error, not a second posting; a row genuinely no longer held and not found in `contract_fees` either reports `NOT_CURRENTLY_HELD`.
9. A `TIE` to an account already tied (by any route) to the *same* `external_transaction_id` reports `already_resolved`; a `TIE` to a *different* account than an existing tie for that posting reports a `CONFLICT`, never a silent success.
10. Editing or deleting `candidate_customer_name` (or any other display-only column) in the uploaded file has no effect on which account is tied or on any validation outcome.
11. A malformed `contract_id`, an unknown account, a `TIE`/`CANDIDATE_REJECT` with `reason_note` under 10 characters, a decision missing its required `contract_id` or carrying one it shouldn't, and an unrecognised `decision` value are each `REJECTED` with that specific reason; none blocks any other row in the file.
12. Two rows sharing the same `external_transaction_id` in one file are both `REJECTED` as duplicates, regardless of file order.
13. A single mixed file — one valid `TIE`, one invalid row, one `DEFER`, one blank/`UNCHANGED` row — processes all four independently and reports four distinct outcomes.
14. `customer_id` does not appear anywhere in the export; `customer_name` appears only as a label next to `candidate_contract_id` and is never read by the import (proved by 10 above).
15. A round-trip test with a leading-zero account number, a long account number, a narration containing a comma/quote/newline, and a narration beginning with `=` proves the file-format and formula-injection rules in section 5 preserve and neutralise every value correctly.

16 A `TIE` whose target account later turns out to already carry a fee for the same loan/type from the workbook or LOS source (section 1.3) re-applies fee source precedence exactly as a screen-driven tie would — the row lands `PENDING` for ordinary classification and review, and any resulting overlap/difference appears on the existing, separate queues (`decideOverlap`), not handled here.

17 Uploading a file that mixes a successful `TIE`, a `CONFLICT`, a `STALE_EVIDENCE` row and a `REJECTED` row writes exactly one `Held Fee Import Processed` audit entry, with the outcome counts and the per-row detail matching the result returned to the uploader — including the rows that did **not** mutate anything.

18 The `TIE`'s own audit entry (the pre-existing one `assignHeld()` writes) carries `meta.channel = 'bulk_import'` and the `import_batch_id` of the upload-level entry from 17; a tie done through the screen in the same test has no `channel` key at all, proving the two are now distinguishable and that the screen's behaviour is unchanged.

19 The same distinction holds for `CANDIDATE_REJECT` against `dismissCandidate()`'s existing audit entry.

9. Out of scope for this item

- No change to `GlFeeBuildService`'s three matching rules (section 1.2).
- No new E-Banker extract or landing-zone question.
- No bulk action for `fee_source_differences` or `source_overlap` (spec 7.8.5, section 1.3) — a separate queue, not touched here.
- **No terminal "not a loan's fee" or "Mega Farm" posting-level disposition.** Section 1.4 established that no such mechanism exists today. If MAIIC wants one, it needs its own governed design — durable storage keyed by `external_transaction_id`, audit, and explicit interaction with the rebuild (so a terminally-excluded posting does not simply reappear as held, or worse, as a false automated match, on the next GL fee build) and with EIR readiness checks. That is a separate spec, not an extension of `dismissCandidate`.

10. Glossary, for a reviewer unfamiliar with this codebase

Term	Meaning
E-Banker	MAIIC's core banking system (Oracle), where loans, payments and the GL are recorded. This system only reads from it.
Batch	One E-Banker posting event; a balanced double-entry batch can have more than two legs. Identified by transaction date, batch type and batch number.
GL_04a / GL_04b	The two extracts this build joins: every fee-GL posting, and every other leg of the same batches.
`contract_id`	The E-Banker loan account number; the key the whole engine, including `contract_fees`, is organised around.
`external_transaction_id`	`'GL_04a:' . CUMVOUCH_DET_ID` — E-Banker's own voucher number for the fee posting. The one durable identity for a held posting across rebuilds.
`contract_fees`	The engine's one table of fees tied to loans, one row per fee per loan, whichever source it came from.
`fee_gl_unassigned`	Fee-GL postings the automatic build could not tie to one loan; **rebuilt from scratch on every run** — its numeric `id` is not durable.
`fee_gl_candidate_dismissals`	Durable record that a specific candidate was rejected for a specific posting; does not remove the posting from the held queue.
Fee source precedence	The governed rule deciding which source "owns" a fee when more than one source holds it for the same loan and type, so it is never counted twice.
Maker-checker	MAIIC's control model: one person proposes or does something, a second person (or the same act's own re-check) must agree before it is final.
O19	The 7 October 2026 decision that the E-Banker system record, not a signed letter or other document, governs the EIR calculation.